MySQL Practical Queries & Technical Questions

What Interviewers Actually Ask

80+ query problems with full solutions,
output tables, and technical Q&A
Written in simple English

Contents

1. How to Use This Book
2. The Practice Schema
3. Basic SELECT & Filtering Queries
4. Aggregate and GROUP BY Queries
5. JOIN Query Problems
6. Subquery Problems
7. Ranking & Nth Highest Problems
8. Duplicate Handling Problems
9. Date and Time Problems
10. String Manipulation Problems
11. Window Function Problems
12. Advanced / Tricky Puzzles
13. Pivot and Report Queries
14. Data Modification Problems
15. Performance Scenario Questions
16. Transaction & Locking Scenarios
17. Database Design Scenarios
18. Rapid-Fire Technical Questions
19. Output Prediction Questions (Tricky)
20. Final Preparation Checklist

1. How to Use This Book

This book is for the **hands-on part** of a MySQL interview — where you are asked to write a query on a whiteboard, in a shared editor, or in an online test.

The right way to practise

1. Read the problem and **write your own query first** on paper. Do not look at the answer.
2. Then compare with the solution. If yours is different but correct, that is fine — there are many correct answers.
3. Read the explanation underneath. The explanation is usually what the interviewer really wants to hear.
4. Repeat the starred problems the day before your interview.

How to answer a query question out loud

Interviewers grade your **thinking**, not only the final query. Say your steps as you write:

- "First I need rows from employees and departments, so I will join them on dept_id."
- "I only want active staff, so that filter goes in WHERE."
- "I need one row per department, so I group by dept_id."
- "The filter on the count must go in HAVING because it is an aggregate."

> **Golden rule:** if you are stuck, ask a clarifying question. "Should employees with no department be included?" is a great question — it tells the interviewer you understand the difference between INNER JOIN and LEFT JOIN.

2. The Practice Schema

Every query in this book uses these tables. Memorise them so you can answer quickly.

```
CREATE TABLE departments (
  dept_id    INT PRIMARY KEY,
  dept_name  VARCHAR(50) NOT NULL,
  city       VARCHAR(50)
);

CREATE TABLE employees (
  emp_id     INT PRIMARY KEY AUTO_INCREMENT,
  first_name VARCHAR(50) NOT NULL,
  last_name  VARCHAR(50),
  email      VARCHAR(120),
  salary     DECIMAL(10,2),
  commission DECIMAL(10,2),          -- can be NULL
  dept_id    INT,
  manager_id INT,                    -- points to employees.emp_id
  hire_date  DATE,
  status     ENUM('active','leave','left') DEFAULT 'active',
  FOREIGN KEY (dept_id)    REFERENCES departments(dept_id),
  FOREIGN KEY (manager_id) REFERENCES employees(emp_id)
);

CREATE TABLE customers (
  cust_id   INT PRIMARY KEY,
  name      VARCHAR(80),
  city      VARCHAR(50),
  signup_date DATE
);

CREATE TABLE orders (
  order_id   INT PRIMARY KEY,
  cust_id    INT,
  order_date DATE,
  amount     DECIMAL(10,2),
  status     VARCHAR(20),
  FOREIGN KEY (cust_id) REFERENCES customers(cust_id)
);

CREATE TABLE products (
  product_id INT PRIMARY KEY,
  name       VARCHAR(80),
  category   VARCHAR(50),
  price      DECIMAL(10,2),
  stock      INT
);

CREATE TABLE order_items (
  order_id   INT,
  product_id INT,
  qty        INT,
  unit_price DECIMAL(10,2),
  PRIMARY KEY (order_id, product_id)
);
```

Sample data used in the examples

emp_id	first_name	salary	commission	dept_id	manager_id	hire_date
1	Ali	120000	NULL	1	NULL	2019-01-10
2	Sara	90000	5000	1	1	2020-03-15
3	Bilal	90000	NULL	2	1	2021-07-01
4	Hina	70000	3000	2	3	2022-02-20
5	Usman	55000	NULL	3	3	2023-09-05
6	Ayesha	55000	NULL	NULL	1	2024-11-11

3. Basic SELECT & Filtering Queries

Q1. Show all employees whose salary is between 60,000 and 100,000.

```
SELECT emp_id, first_name, salary
FROM employees
WHERE salary BETWEEN 60000 AND 100000;
```

Note: BETWEEN is **inclusive** of both ends. It is the same as `salary >= 60000 AND salary <= 100000`.

Q2. Find employees whose name starts with 'A' and has exactly 3 letters.

```
SELECT * FROM employees
WHERE first_name LIKE 'A__';
```

Note: `%` matches any number of characters, `_` matches exactly one. `'A__'` means A plus two more characters.

Q3. List employees who have no commission. ★

```
SELECT * FROM employees WHERE commission IS NULL;
```

Common mistake: writing `WHERE commission = NULL`. That always returns nothing, because anything compared to NULL gives NULL (unknown), not true. You must use `IS NULL`.

Q4. Show total pay (salary + commission) for each employee.

```
SELECT first_name,
       salary + IFNULL(commission, 0) AS total_pay
FROM employees;
```

Why IFNULL: `90000 + NULL = NULL`. Without IFNULL, every employee with no commission shows NULL instead of their salary. This is a very popular trap question.

Q5. Show the 3rd to 5th highest paid employees (pagination).

```
SELECT first_name, salary
FROM employees
ORDER BY salary DESC
LIMIT 3 OFFSET 2;
```

Note: OFFSET skips rows, LIMIT takes rows. `LIMIT 3 OFFSET 2` skips the top 2 and returns the next 3. MySQL also allows the short form `LIMIT 2, 3` (offset first).

Q6. Sort employees by department, and inside each department by salary (highest first).

```
SELECT * FROM employees
ORDER BY dept_id ASC, salary DESC;
```

Q7. Put NULL departments at the end of the sort. ★

```
SELECT * FROM employees
ORDER BY (dept_id IS NULL), dept_id;
```

Explanation: MySQL sorts NULL first by default in ASC. `dept_id IS NULL` returns 0 or 1, so rows with NULL get 1 and go last.

Q8. Show employees from department 1, 3 or 5 who are active.

```sql
SELECT * FROM employees
WHERE dept_id IN (1, 3, 5)
  AND status = 'active';
```

Q9. Give each employee a grade based on salary.

```sql
SELECT first_name, salary,
  CASE
    WHEN salary >= 100000 THEN 'A'
    WHEN salary >= 70000  THEN 'B'
    WHEN salary >= 50000  THEN 'C'
    ELSE 'D'
  END AS grade
FROM employees;
```

Note: CASE checks conditions **top to bottom** and stops at the first match, so order them from highest to lowest.

Q10. Count how many employees are active vs left, in one row.

```sql
SELECT
  SUM(status = 'active') AS active_count,
  SUM(status = 'left')   AS left_count,
  COUNT(*)               AS total
FROM employees;
```

Trick worth remembering: in MySQL a condition returns 1 or 0, so `SUM(condition)` is a neat conditional count. The portable version is `SUM(CASE WHEN status='active' THEN 1 ELSE 0 END)`.

4. Aggregate and GROUP BY Queries

Q11. Find the average salary of each department.

```
SELECT dept_id, AVG(salary) AS avg_salary, COUNT(*) AS headcount
FROM employees
GROUP BY dept_id;
```

Q12. Show only departments where the average salary is above 80,000. ★

```
SELECT dept_id, AVG(salary) AS avg_salary
FROM employees
GROUP BY dept_id
HAVING AVG(salary) > 80000;
```

Key point the interviewer wants: you cannot use WHERE here, because WHERE runs before grouping and cannot see aggregate values. Filters on aggregates always go in HAVING.

Q13. Show active employees only, grouped by department, keeping groups with more than 2 people.

```
SELECT dept_id, COUNT(*) AS c
FROM employees
WHERE status = 'active'     -- filters rows BEFORE grouping
GROUP BY dept_id
HAVING COUNT(*) > 2;        -- filters groups AFTER grouping
```

This one query proves you understand WHERE vs HAVING. Use it as your standard answer.

Q14. What is the difference in output of COUNT(*), COUNT(commission) and COUNT(DISTINCT salary)? ★

```
SELECT COUNT(*)                AS all_rows,        -- 6
       COUNT(commission)       AS with_commission, -- 2
       COUNT(DISTINCT salary)  AS distinct_salaries-- 4
FROM employees;
```

Explanation with the sample data: 6 rows total; only Sara and Hina have a commission; the salaries 120000, 90000, 90000, 70000, 55000, 55000 contain 4 different values.

Q15. Find the department with the highest total salary bill.

```
SELECT dept_id, SUM(salary) AS total_bill
FROM employees
GROUP BY dept_id
ORDER BY total_bill DESC
LIMIT 1;
```

Q16. Show department-wise counts plus a grand total row.

```
SELECT IFNULL(CAST(dept_id AS CHAR), 'TOTAL') AS department,
       COUNT(*) AS headcount
FROM employees
GROUP BY dept_id WITH ROLLUP;
```

Note: WITH ROLLUP adds a summary row where the grouped column is NULL.

Q17. List all employee names of each department in a single comma-separated string. ★

```sql
SELECT dept_id,
       GROUP_CONCAT(first_name ORDER BY first_name SEPARATOR ', ') AS team
FROM employees
GROUP BY dept_id;
```

Watch out: GROUP_CONCAT is cut off at 1024 characters by default. Raise it with `SET SESSION group_concat_max_len = 100000;` — mentioning this scores extra points.

Q18. Find the minimum, maximum and salary range per department.

```sql
SELECT dept_id,
       MIN(salary) AS lowest,
       MAX(salary) AS highest,
       MAX(salary) - MIN(salary) AS salary_range
FROM employees
GROUP BY dept_id;
```

Q19. Count employees hired in each year.

```sql
SELECT YEAR(hire_date) AS yr, COUNT(*) AS hires
FROM employees
GROUP BY YEAR(hire_date)
ORDER BY yr;
```

Performance note: `YEAR(hire_date)` in GROUP BY is fine, but the same expression in a WHERE clause would block index use. Say this if they ask about performance.

Q20. Calculate the percentage of total salary each department takes. ★

```sql
SELECT dept_id,
       SUM(salary) AS dept_total,
       ROUND(SUM(salary) * 100.0 / (SELECT SUM(salary) FROM employees), 2)
         AS percent_of_total
FROM employees
GROUP BY dept_id;
```

Modern alternative with a window function:

```sql
SELECT DISTINCT dept_id,
       SUM(salary) OVER (PARTITION BY dept_id) * 100.0
       / SUM(salary) OVER () AS pct
FROM employees;
```

5. JOIN Query Problems

Q21. Show each employee with their department name.

```
SELECT e.first_name, d.dept_name
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id;
```

Follow-up they will ask: "Ayesha has no department — does she appear?" No. INNER JOIN drops unmatched rows. Use LEFT JOIN to keep her.

Q22. Show all employees, even those with no department. ★

```
SELECT e.first_name, IFNULL(d.dept_name, 'Unassigned') AS department
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id;
```

Q23. Find departments that have no employees (anti-join). ★★

```
SELECT d.dept_id, d.dept_name
FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id
WHERE e.emp_id IS NULL;
```

Explanation: the LEFT JOIN keeps every department; where there is no matching employee, all employee columns come back NULL. Filtering on `e.emp_id IS NULL` therefore leaves only the empty departments. This "LEFT JOIN + IS NULL" pattern is one of the most asked interview patterns.

Alternative:

```
SELECT * FROM departments d
WHERE NOT EXISTS (SELECT 1 FROM employees e WHERE e.dept_id = d.dept_id);
```

Q24. Show each employee next to their manager's name (self join). ★★

```
SELECT e.first_name AS employee,
       IFNULL(m.first_name, 'No Manager') AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.emp_id;
```

Explanation: the same table is used twice with two different aliases: `e` is the worker, `m` is the manager. LEFT JOIN is used so the CEO (manager_id NULL) is not lost.

Q25. Find employees who earn more than their manager. ★★

```
SELECT e.first_name AS employee, e.salary,
       m.first_name AS manager, m.salary AS manager_salary
FROM employees e
JOIN employees m ON e.manager_id = m.emp_id
WHERE e.salary > m.salary;
```

This is a famous LeetCode-style question. The trick is realising it is a self join plus a simple comparison.

Q26. Count how many people report to each manager (include managers with zero reports).

```
SELECT m.emp_id, m.first_name AS manager,
       COUNT(e.emp_id) AS team_size
FROM employees m
LEFT JOIN employees e ON e.manager_id = m.emp_id
GROUP BY m.emp_id, m.first_name;
```

Important: use `COUNT(e.emp_id)`, not `COUNT(*)`. With LEFT JOIN, a manager with no team still produces one row, and `COUNT(*)` would wrongly report 1 instead of 0.

Q27. In a LEFT JOIN, why does adding a WHERE condition on the right table break it? ★★

```
-- WRONG: this behaves like an INNER JOIN
SELECT e.first_name, d.dept_name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id
WHERE d.city = 'Lahore';

-- RIGHT: put the condition in ON
SELECT e.first_name, d.dept_name
FROM employees e
LEFT JOIN departments d
  ON e.dept_id = d.dept_id AND d.city = 'Lahore';
```

Explanation: the LEFT JOIN first produces NULLs for unmatched rows. Then WHERE tests `NULL = 'Lahore'`, which is not true, so those rows are removed — cancelling the whole point of the LEFT JOIN. Conditions on the optional table belong in ON.

Q28. Show customers and their total order amount, including customers who never ordered.

```
SELECT c.cust_id, c.name,
       COUNT(o.order_id)      AS order_count,
       IFNULL(SUM(o.amount),0) AS total_spent
FROM customers c
LEFT JOIN orders o ON c.cust_id = o.cust_id
GROUP BY c.cust_id, c.name
ORDER BY total_spent DESC;
```

Q29. Simulate a FULL OUTER JOIN in MySQL. ★

```
SELECT e.first_name, d.dept_name
FROM employees e LEFT JOIN departments d ON e.dept_id = d.dept_id
UNION
SELECT e.first_name, d.dept_name
FROM employees e RIGHT JOIN departments d ON e.dept_id = d.dept_id;
```

Say this: "MySQL does not support FULL OUTER JOIN, so I combine a LEFT JOIN and a RIGHT JOIN with UNION, which also removes the duplicated matching rows."

Q30. Join three tables: order details with customer and product names.

```
SELECT o.order_id, c.name AS customer, p.name AS product,
       oi.qty, oi.unit_price, oi.qty * oi.unit_price AS line_total
FROM orders o
JOIN customers   c  ON o.cust_id    = c.cust_id
JOIN order_items oi ON o.order_id   = oi.order_id
JOIN products    p  ON oi.product_id = p.product_id
ORDER BY o.order_id;
```

Q31. What is a CROSS JOIN and when is it useful?

```
SELECT d.dept_name, m.month_name
FROM departments d
CROSS JOIN months m;
```

Answer: it pairs every row of A with every row of B. If A has 10 rows and B has 12, you get 120. It is useful to build a complete grid for reports — for example every department × every month — so months with no data still appear as zero instead of being missing. Accidental cross joins (forgetting the ON clause) are a classic bug.

6. Subquery Problems

Q32. Find employees earning more than the company average. ★

```
SELECT first_name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Why a subquery is required: you cannot write `WHERE salary > AVG(salary)` — aggregate functions are not allowed in WHERE.

Q33. Find employees earning more than the average of their own department. ★★

```
-- correlated subquery version
SELECT e.first_name, e.dept_id, e.salary
FROM employees e
WHERE e.salary > (SELECT AVG(salary) FROM employees
                  WHERE dept_id = e.dept_id);

-- faster join version
SELECT e.first_name, e.salary, a.avg_sal
FROM employees e
JOIN (SELECT dept_id, AVG(salary) AS avg_sal
      FROM employees GROUP BY dept_id) a
  ON e.dept_id = a.dept_id
WHERE e.salary > a.avg_sal;
```

Say this: "The first is a correlated subquery — it runs once per outer row. The second computes each department average once and joins, so it usually performs better on large tables."

Q34. Find customers who have placed at least one order (three ways).

```
-- 1. EXISTS (usually best)
SELECT * FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.cust_id = c.cust_id);

-- 2. IN
SELECT * FROM customers
WHERE cust_id IN (SELECT cust_id FROM orders);

-- 3. JOIN with DISTINCT
SELECT DISTINCT c.* FROM customers c
JOIN orders o ON c.cust_id = o.cust_id;
```

Q35. Why can NOT IN give wrong results? ★★

```
-- DANGEROUS: returns ZERO rows if any cust_id in orders is NULL
SELECT * FROM customers
WHERE cust_id NOT IN (SELECT cust_id FROM orders);

-- SAFE
SELECT * FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.cust_id = c.cust_id);
```

Explanation: `x NOT IN (1, 2, NULL)` becomes `x<>1 AND x<>2 AND x<>NULL` . The last part is NULL (unknown), so the whole condition can never be true. NOT EXISTS has no such problem. Knowing this single fact impresses interviewers.

Q36. Find the employee(s) with the maximum salary.

```
SELECT * FROM employees
WHERE salary = (SELECT MAX(salary) FROM employees);
```

Why not ORDER BY ... LIMIT 1? Because if two people tie for the top salary, LIMIT 1 shows only one of them. The subquery version returns all of them. Mention this trade-off.

Q37. Show each department with the name of its highest paid employee. ★★

```
SELECT e.dept_id, e.first_name, e.salary
FROM employees e
WHERE e.salary = (SELECT MAX(salary) FROM employees
                  WHERE dept_id = e.dept_id);

-- MySQL 8 window function version (cleaner)
SELECT dept_id, first_name, salary FROM (
  SELECT *, RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) rnk
  FROM employees
) t WHERE rnk = 1;
```

Q38. Use a derived table to find departments whose average salary beats 70,000.

```
SELECT * FROM (
  SELECT dept_id, AVG(salary) AS avg_sal
  FROM employees GROUP BY dept_id
) AS t
WHERE t.avg_sal > 70000;
```

Rule: a subquery in FROM (a derived table) **must** have an alias, otherwise MySQL raises "Every derived table must have its own alias".

7. Ranking & Nth Highest Problems

Q39. Find the 2nd highest salary. ★★★

Method 1 — LIMIT/OFFSET (simplest):

```
SELECT DISTINCT salary FROM employees
ORDER BY salary DESC LIMIT 1 OFFSET 1;
```

Method 2 — subquery (works in old MySQL):

```
SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Method 3 — window function (best answer in MySQL 8):

```
SELECT DISTINCT salary FROM (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
  FROM employees
) t WHERE rnk = 2;
```

Interview gold: mention that `DISTINCT` / `DENSE_RANK` matters because of duplicate salaries. With salaries 120000, 90000, 90000, the second highest is 90000, not 90000-the-second-row.

Q40. Find the Nth highest salary as a reusable function.

```
DELIMITER $$
CREATE FUNCTION nth_highest(n INT) RETURNS DECIMAL(10,2)
DETERMINISTIC READS SQL DATA
BEGIN
  DECLARE result DECIMAL(10,2);
  SELECT DISTINCT salary INTO result FROM employees
  ORDER BY salary DESC LIMIT 1 OFFSET 0;  -- see note
  RETURN result;
END$$
DELIMITER ;
```

Note: LIMIT does not accept a variable directly in older versions; use `SET n = n - 1;` with a prepared statement, or simply use DENSE_RANK inside the function. Interviewers accept the DENSE_RANK version.

Q41. Explain the difference between ROW_NUMBER, RANK and DENSE_RANK. ★★★

```
SELECT salary,
  ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num,
  RANK()       OVER (ORDER BY salary DESC) AS rnk,
  DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rnk
FROM employees;
```

salary	ROW_NUMBER	RANK	DENSE_RANK
120000	1	1	1
90000	2	2	2
90000	3	2	2
70000	4	4	3
55000	5	5	4

One-line memory hook: ROW_NUMBER never ties; RANK ties and skips; DENSE_RANK ties and does not skip.

Q42. Get the top 2 earners in every department. ★★

```sql
SELECT dept_id, first_name, salary
FROM (
  SELECT dept_id, first_name, salary,
         DENSE_RANK() OVER (PARTITION BY dept_id
                            ORDER BY salary DESC) AS rnk
  FROM employees
) t
WHERE rnk <= 2
ORDER BY dept_id, salary DESC;
```

Why PARTITION BY: it restarts the ranking for each department instead of ranking the whole company.

Q43. Find the median salary. ★

```sql
SELECT AVG(salary) AS median_salary FROM (
  SELECT salary,
         ROW_NUMBER() OVER (ORDER BY salary) AS rn,
         COUNT(*) OVER () AS total
  FROM employees
) t
WHERE rn IN (FLOOR((total+1)/2), CEIL((total+1)/2));
```

Explanation: MySQL has no MEDIAN function. You number the rows in order and pick the middle one; for an even count you take the average of the two middle values.

8. Duplicate Handling Problems

Q44. Find duplicate email addresses. ★★

```
SELECT email, COUNT(*) AS occurrences
FROM employees
GROUP BY email
HAVING COUNT(*) > 1;
```

Q45. Show the full rows of all duplicates, not just the email.

```
SELECT * FROM employees
WHERE email IN (
  SELECT email FROM employees GROUP BY email HAVING COUNT(*) > 1
);

-- window function version
SELECT * FROM (
  SELECT *, COUNT(*) OVER (PARTITION BY email) AS c FROM employees
) t WHERE c > 1;
```

Q46. Delete duplicate rows but keep the one with the smallest id. ★★★

```
DELETE e1 FROM employees e1
JOIN employees e2
  ON e1.email = e2.email
 AND e1.emp_id > e2.emp_id;
```

Explanation: the join pairs each duplicate with the copy that has a smaller id. Any row that has such a partner is deleted, so only the smallest id of each email survives.

Window function version (MySQL 8):

```
DELETE FROM employees WHERE emp_id IN (
  SELECT emp_id FROM (
    SELECT emp_id, ROW_NUMBER() OVER (PARTITION BY email
                                      ORDER BY emp_id) rn
    FROM employees
  ) t WHERE rn > 1
);
```

Note: the extra derived table is needed because MySQL will not let you select from the same table you are deleting from directly.

Q47. Prevent duplicates from happening in the future.

```
ALTER TABLE employees ADD CONSTRAINT uq_email UNIQUE (email);
```

Good extra answer: "Fixing existing duplicates is only half the job — I would add a UNIQUE constraint so the database itself prevents them, rather than relying on application code."

9. Date and Time Problems

Q48. Find employees hired in the last 30 days.

```
SELECT * FROM employees
WHERE hire_date >= CURDATE() - INTERVAL 30 DAY;
```

Q49. Calculate each employee's years of service.

```
SELECT first_name,
       TIMESTAMPDIFF(YEAR, hire_date, CURDATE()) AS years_of_service
FROM employees;
```

Why not DATEDIFF/365? TIMESTAMPDIFF handles leap years correctly. DATEDIFF gives days, which you would have to divide and round yourself.

Q50. Find all orders placed in the current month.

```
SELECT * FROM orders
WHERE order_date >= DATE_FORMAT(CURDATE(), '%Y-%m-01')
  AND order_date <  DATE_FORMAT(CURDATE(), '%Y-%m-01') + INTERVAL 1 MONTH;
```

Why written this way: using `WHERE MONTH(order_date) = MONTH(CURDATE())` works but wraps the column in a function, so the index on order_date cannot be used. The range version is index-friendly. **This is a top performance answer.**

Q51. Show monthly sales totals for the last 12 months.

```
SELECT DATE_FORMAT(order_date, '%Y-%m') AS month,
       COUNT(*)    AS orders,
       SUM(amount) AS revenue
FROM orders
WHERE order_date >= CURDATE() - INTERVAL 12 MONTH
GROUP BY DATE_FORMAT(order_date, '%Y-%m')
ORDER BY month;
```

Q52. Find employees whose birthday (or work anniversary) is this month.

```
SELECT * FROM employees
WHERE MONTH(hire_date) = MONTH(CURDATE());
```

Q53. Find the number of days between an order and its delivery, ignoring weekends conceptually.

```
SELECT order_id,
       DATEDIFF(delivered_date, order_date) AS days_taken,
       DAYNAME(order_date) AS ordered_on
FROM orders;
```

Q54. Show the first and last day of the current month.

```
SELECT DATE_FORMAT(CURDATE(), '%Y-%m-01') AS first_day,
       LAST_DAY(CURDATE())                AS last_day;
```

10. String Manipulation Problems

Q55. Show full names in "LAST, First" format.

```
SELECT CONCAT(UPPER(last_name), ', ', first_name) AS full_name
FROM employees;
```

Careful: `CONCAT` returns NULL if any argument is NULL. Use `CONCAT_WS(', ', last_name, first_name)`, which skips NULLs instead.

Q56. Extract the domain from an email address. ★

```
SELECT email,
       SUBSTRING_INDEX(email, '@', -1) AS domain,
       SUBSTRING_INDEX(email, '@',  1) AS username
FROM employees;
```

Explanation: a positive count takes text before the Nth separator; a negative count takes text after it, counting from the right.

Q57. Find names containing a specific word, case-insensitively.

```
SELECT * FROM employees WHERE LOWER(first_name) LIKE '%ali%';
```

Note: MySQL's default collation is already case-insensitive (`..._ci`), so `LIKE '%ali%'` alone usually matches "Ali" too. Mention collation — it shows depth.

Q58. Mask an email for privacy: show only the first 2 characters.

```
SELECT CONCAT(LEFT(email, 2), '****@',
              SUBSTRING_INDEX(email, '@', -1)) AS masked
FROM employees;
```

Q59. Remove extra spaces and standardise capitalisation.

```
SELECT CONCAT(
         UPPER(LEFT(TRIM(first_name), 1)),
         LOWER(SUBSTRING(TRIM(first_name), 2))
       ) AS proper_name
FROM employees;
```

Note: MySQL has no built-in INITCAP / proper-case function, so you build it manually. Saying that out loud is a good answer.

Q60. Find rows where a column has leading or trailing spaces (a data-quality check).

```
SELECT * FROM employees WHERE first_name <> TRIM(first_name);
```

11. Window Function Problems

Q61. Show each employee's salary alongside their department average, without collapsing rows. ★★

```
SELECT first_name, dept_id, salary,
       AVG(salary) OVER (PARTITION BY dept_id) AS dept_avg,
       salary - AVG(salary) OVER (PARTITION BY dept_id) AS diff
FROM employees;
```

The key sentence: "GROUP BY would reduce each department to one row. PARTITION BY keeps every row and just calculates the average per department, which is exactly what is needed here."

Q62. Calculate a running total of daily sales. ★★

```
SELECT order_date, amount,
       SUM(amount) OVER (ORDER BY order_date
                         ROWS BETWEEN UNBOUNDED PRECEDING
                         AND CURRENT ROW) AS running_total
FROM orders
ORDER BY order_date;
```

Q63. Compare each month's revenue with the previous month (LAG). ★★

```
SELECT month, revenue,
       LAG(revenue) OVER (ORDER BY month) AS prev_month,
       revenue - LAG(revenue) OVER (ORDER BY month) AS change,
       ROUND((revenue - LAG(revenue) OVER (ORDER BY month))
             * 100.0 / LAG(revenue) OVER (ORDER BY month), 2) AS pct_change
FROM (
  SELECT DATE_FORMAT(order_date,'%Y-%m') AS month, SUM(amount) AS revenue
  FROM orders GROUP BY 1
) m;
```

Note: the first row has no previous month, so LAG returns NULL and the change is NULL. Wrap with `IFNULL` if you want 0.

Q64. Split employees into 4 salary quartiles.

```
SELECT first_name, salary,
       NTILE(4) OVER (ORDER BY salary DESC) AS quartile
FROM employees;
```

Q65. Show a 3-day moving average of sales.

```
SELECT order_date, amount,
       AVG(amount) OVER (ORDER BY order_date
                         ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)
         AS moving_avg_3day
FROM orders;
```

Q66. Show the highest and lowest salary in each department on every row.

```
SELECT first_name, dept_id, salary,
       FIRST_VALUE(salary) OVER (PARTITION BY dept_id
                                 ORDER BY salary DESC) AS dept_max,
       MIN(salary) OVER (PARTITION BY dept_id) AS dept_min
FROM employees;
```

12. Advanced / Tricky Puzzles

Q67. Find consecutive numbers appearing at least 3 times in a row (classic puzzle). ★★

```
SELECT DISTINCT l1.num AS consecutive_num
FROM logs l1
JOIN logs l2 ON l1.id = l2.id - 1 AND l1.num = l2.num
JOIN logs l3 ON l1.id = l3.id - 2 AND l1.num = l3.num;

-- MySQL 8 version with LAG/LEAD
SELECT DISTINCT num FROM (
  SELECT num,
         LAG(num)  OVER (ORDER BY id) AS prev,
         LEAD(num) OVER (ORDER BY id) AS nxt
  FROM logs
) t WHERE num = prev AND num = nxt;
```

Q68. Find gaps in a sequence of IDs. ★

```
SELECT id + 1 AS gap_starts_at,
       next_id - 1 AS gap_ends_at
FROM (
  SELECT id, LEAD(id) OVER (ORDER BY id) AS next_id FROM orders
) t
WHERE next_id - id > 1;
```

Q69. Swap two column values without a temporary column.

```
UPDATE employees SET salary = commission, commission = salary;
```

Important: this works in MySQL only if the assignments are evaluated correctly — MySQL evaluates left to right, so the safe portable version is:

```
UPDATE employees
SET salary = salary + commission,
    commission = salary - commission,
    salary = salary - commission;
```

Or simply use a temporary variable. Interviewers ask this to see whether you know UPDATE evaluates assignments in order.

Q70. Swap 'm' and 'f' in a gender column in one statement. ★

```
UPDATE salary SET sex = IF(sex = 'm', 'f', 'm');

-- or
UPDATE salary SET sex = CASE sex WHEN 'm' THEN 'f' ELSE 'm' END;
```

Q71. Find customers who bought product A but not product B. ★★

```
SELECT DISTINCT o.cust_id
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
WHERE oi.product_id = 1
  AND o.cust_id NOT IN (
      SELECT o2.cust_id FROM orders o2
      JOIN order_items oi2 ON o2.order_id = oi2.order_id
      WHERE oi2.product_id = 2
  );
```

Q72. Find customers who bought all products (relational division). ★★★

```sql
SELECT o.cust_id
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY o.cust_id
HAVING COUNT(DISTINCT oi.product_id) = (SELECT COUNT(*) FROM products);
```

Explanation: count the distinct products each customer bought and compare it with the total number of products. This is the standard trick for "bought all / took every course / worked in every department".

Q73. Find employees who work in all departments where their manager works.

```sql
-- generalised pattern: NOT EXISTS ( ... NOT EXISTS ... )
SELECT e.emp_id FROM employees e
WHERE NOT EXISTS (
  SELECT 1 FROM departments d
  WHERE NOT EXISTS (
    SELECT 1 FROM employees x
    WHERE x.emp_id = e.emp_id AND x.dept_id = d.dept_id
  )
);
```

Read it as: "there is no department for which this employee has no record" = "the employee is in every department".

Q74. Generate a list of numbers 1 to 100 with no table.

```sql
WITH RECURSIVE nums AS (
  SELECT 1 AS n
  UNION ALL
  SELECT n + 1 FROM nums WHERE n < 100
)
SELECT * FROM nums;
```

Q75. Print the full management hierarchy with levels. ★★

```sql
WITH RECURSIVE org AS (
  SELECT emp_id, first_name, manager_id, 1 AS lvl,
         CAST(first_name AS CHAR(500)) AS path
  FROM employees WHERE manager_id IS NULL
  UNION ALL
  SELECT e.emp_id, e.first_name, e.manager_id, o.lvl + 1,
         CONCAT(o.path, ' > ', e.first_name)
  FROM employees e
  JOIN org o ON e.manager_id = o.emp_id
)
SELECT lvl, path FROM org ORDER BY path;
```

Structure of any recursive CTE: an anchor query (the starting rows), UNION ALL, then a recursive query that joins back to the CTE, and a stopping condition.

Q76. Find the department with the second-most employees.

```sql
SELECT dept_id, headcount FROM (
  SELECT dept_id, COUNT(*) AS headcount,
         DENSE_RANK() OVER (ORDER BY COUNT(*) DESC) AS rnk
  FROM employees GROUP BY dept_id
) t WHERE rnk = 2;
```

Notice: you can use an aggregate inside a window function's ORDER BY, because window functions run after grouping.

13. Pivot and Report Queries

Q77. Pivot: show headcount per status as columns. ★★

```
SELECT dept_id,
  SUM(status = 'active') AS active,
  SUM(status = 'leave')  AS on_leave,
  SUM(status = 'left')   AS resigned,
  COUNT(*)               AS total
FROM employees
GROUP BY dept_id;
```

Say this: "MySQL has no PIVOT keyword, so I pivot manually with conditional aggregation — SUM or COUNT wrapped around a CASE or a boolean condition."

Q78. Monthly revenue by category, months as columns.

```
SELECT p.category,
  SUM(CASE WHEN MONTH(o.order_date)=1 THEN oi.qty*oi.unit_price ELSE 0 END) AS jan,
  SUM(CASE WHEN MONTH(o.order_date)=2 THEN oi.qty*oi.unit_price ELSE 0 END) AS feb,
  SUM(CASE WHEN MONTH(o.order_date)=3 THEN oi.qty*oi.unit_price ELSE 0 END) AS mar
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p     ON oi.product_id = p.product_id
GROUP BY p.category;
```

Q79. Unpivot: turn three phone columns into rows.

```
SELECT emp_id, 'home' AS type, home_phone AS phone FROM contacts
UNION ALL
SELECT emp_id, 'work', work_phone FROM contacts
UNION ALL
SELECT emp_id, 'mobile', mobile FROM contacts;
```

14. Data Modification Problems

Q80. Give a 10% raise to everyone in the Engineering department.

```
UPDATE employees e
JOIN departments d ON e.dept_id = d.dept_id
SET e.salary = e.salary * 1.10
WHERE d.dept_name = 'Engineering';
```

Q81. Insert a row, or update it if the key already exists (upsert). ★

```
INSERT INTO products (product_id, name, stock)
VALUES (10, 'Keyboard', 50)
ON DUPLICATE KEY UPDATE stock = stock + VALUES(stock);
```

Compare with REPLACE: REPLACE deletes the old row and inserts a new one, so unlisted columns reset to defaults and DELETE triggers fire. `ON DUPLICATE KEY UPDATE` is usually the safer choice.

Q82. Copy last year's orders into an archive table and delete them.

```
START TRANSACTION;
  INSERT INTO orders_archive
  SELECT * FROM orders WHERE order_date < '2026-01-01';

  DELETE FROM orders WHERE order_date < '2026-01-01';
COMMIT;
```

Why a transaction: if the DELETE fails after the INSERT succeeded, you would have the rows twice. Wrapping both in one transaction makes it all-or-nothing.

Q83. Update a column using values from another table.

```
UPDATE products p
JOIN (SELECT product_id, SUM(qty) AS sold
      FROM order_items GROUP BY product_id) s
  ON p.product_id = s.product_id
SET p.stock = p.stock - s.sold;
```

Q84. Delete the 5 oldest log rows.

```
DELETE FROM logs ORDER BY created_at ASC LIMIT 5;
```

15. Performance Scenario Questions

S1. "This query takes 30 seconds. How would you fix it?"
Answer in steps: (1) run `EXPLAIN` and look at the `type` column — if it says `ALL`, there is a full table scan; (2) check `key` to see whether an index was used; (3) add an index on the columns in WHERE/JOIN/ORDER BY; (4) remove functions wrapped around indexed columns; (5) select fewer columns; (6) check `Extra` for "Using filesort" or "Using temporary"; (7) run `ANALYZE TABLE` so statistics are fresh; (8) consider a covering index, caching, or partitioning for very large tables.

S2. "You added an index but the query got slower overall. Why?"
Because indexes speed up reads but slow down every INSERT, UPDATE and DELETE, since the index must be maintained too. On a write-heavy table, too many indexes hurt. Also, the optimizer may ignore a poorly selective index, so you pay the write cost with no read benefit.

S3. "Which columns would you index in this query?"
`SELECT name FROM employees WHERE dept_id = 5 AND status = 'active' ORDER BY salary DESC;`
A composite index on `(dept_id, status, salary)`. The equality columns come first, then the sorting column, which lets MySQL avoid a filesort. Adding `name` would make it a covering index.

S4. "Why is LIMIT 1000000, 10 slow?"
MySQL still has to read and throw away the first million rows. Fix it with keyset pagination: `WHERE id > :last_seen_id ORDER BY id LIMIT 10`, which jumps straight to the right place using the index.

S5. "What is a covering index?"
An index that contains every column a query needs, so MySQL answers entirely from the index and never reads the table. EXPLAIN shows "Using index". It is one of the biggest wins available.

S6. "SELECT COUNT(*) on a huge InnoDB table is slow. Why?"
InnoDB does not store a row count, because with MVCC different transactions can see different numbers of rows, so it must scan an index. MyISAM stores the count and answers instantly. Workarounds: use an approximate count from `SHOW TABLE STATUS`, or maintain a counter table.

S7. "How do you find slow queries in production?"
Enable the **slow query log** with a `long_query_time` threshold, analyse it with `mysqldumpslow` or `pt-query-digest`, and query `performance_schema.events_statements_summary_by_digest` to see the most expensive statements.

S8. "When would a full table scan actually be better than an index?"
When the query returns a large share of the rows (often more than about 20–30%), or the table is small. Reading sequentially is cheaper than jumping between index and table for most rows.

16. Transaction & Locking Scenarios

T1. "Write a safe bank transfer."

```
START TRANSACTION;
  SELECT balance FROM accounts WHERE id = 1 FOR UPDATE;
  UPDATE accounts SET balance = balance - 5000 WHERE id = 1;
  UPDATE accounts SET balance = balance + 5000 WHERE id = 2;
COMMIT;
```

`FOR UPDATE` locks the row so another transaction cannot read-and-overwrite the balance at the same time (a lost update).

T2. "Two users book the last seat at the same time. How do you prevent double booking?"
Either pessimistic locking (`SELECT ... FOR UPDATE` on the seat row inside a transaction), or optimistic locking (`UPDATE seats SET booked=1 WHERE id=? AND booked=0` and check that `ROW_COUNT()` is 1). A UNIQUE constraint on the booking row is the final safety net.

T3. "You get error 1213 deadlock found. What now?"
InnoDB already rolled one transaction back. The application should catch the error and retry. To reduce deadlocks: access rows in the same order everywhere, keep transactions short, index the columns used in WHERE so fewer rows are locked, and inspect `SHOW ENGINE INNODB STATUS` for the last deadlock detail.

T4. "What isolation level would you choose for a reporting system?"
READ COMMITTED is often enough and reduces locking, or you can read from a replica. For financial consistency inside one report, REPEATABLE READ (the MySQL default) guarantees a stable snapshot for the whole transaction.

T5. "Can you roll back a TRUNCATE?"
No. TRUNCATE is DDL and causes an implicit commit. Use `DELETE` inside a transaction if you may need to undo it.

T6. "Build a job queue where two workers never take the same job."

```
START TRANSACTION;
  SELECT * FROM jobs WHERE status='pending'
  ORDER BY id LIMIT 1
  FOR UPDATE SKIP LOCKED;
  UPDATE jobs SET status='processing' WHERE id = ?;
COMMIT;
```

`SKIP LOCKED` (MySQL 8) makes each worker skip rows another worker already locked instead of waiting.

17. Database Design Scenarios

D1. "Design a schema for a blog with posts, authors, comments and tags."

```
authors(author_id PK, name, email UNIQUE)
posts(post_id PK, author_id FK, title, body, published_at)
comments(comment_id PK, post_id FK, user_name, body, created_at)
tags(tag_id PK, name UNIQUE)
post_tags(post_id FK, tag_id FK, PRIMARY KEY(post_id, tag_id))
```

Point out the relationships: author–posts is one-to-many, posts–tags is many-to-many so it needs the junction table `post_tags`.

D2. "How do you model a many-to-many relationship?"
With a third junction table holding two foreign keys, and usually a composite primary key of both. Example: students, courses, and an enrollments table with `(student_id, course_id)` plus extra columns like enrollment_date or grade.

D3. "Should the primary key be an auto-increment INT or a UUID?"
Auto-increment INT is smaller, sequential (so inserts append to the end of the clustered index), and keeps every secondary index small. UUIDs avoid collisions across distributed systems but are larger and random, which causes page splits. A compromise is a binary(16) UUID or an ordered UUID.

D4. "Would you store images in the database?"
Usually no. Store the file in object storage or the filesystem and keep only the path/URL in the database. BLOBs bloat backups, fill the buffer pool, and slow replication.

D5. "How would you store a user's multiple phone numbers?"
Not as phone1, phone2, phone3 — that breaks 1NF and is hard to extend. Create a separate `phones(phone_id, user_id, type, number)` table.

D6. "When would you denormalize?"
When a read-heavy report repeatedly joins many tables and profiling proves the joins are the bottleneck. For example, storing `order_count` on the customer row. Accept that you must now keep it in sync with triggers, application code or a scheduled job.

D7. "The table has 500 million rows and is slow. What options do you have?"
Proper indexing first; then archiving old data, partitioning by date, using summary/aggregate tables for reports, read replicas for read load, and finally sharding if writes are the bottleneck.

18. Rapid-Fire Technical Questions

Question	Short answer
Default MySQL port?	3306
Default storage engine?	InnoDB
Default isolation level?	REPEATABLE READ
Which clause filters groups?	HAVING
Which runs first, WHERE or GROUP BY?	WHERE
Can you use an alias in WHERE?	No (SELECT runs later). You can in ORDER BY / HAVING.
Which join returns unmatched left rows?	LEFT JOIN
Does MySQL support FULL OUTER JOIN?	No — simulate with UNION
Does MySQL support materialized views?	No — simulate with a table plus an event
Data type for money?	DECIMAL
Difference between utf8 and utf8mb4?	utf8mb4 is real 4-byte UTF-8 and supports emoji
Which index type is used by default?	B+ Tree
How many primary keys per table?	One (possibly on several columns)
Can a unique key be NULL?	Yes, and multiple NULLs are allowed
What does TRUNCATE do to AUTO_INCREMENT?	Resets it to 1
Is DDL transactional?	No — implicit commit
What replaced the query cache in 8.0?	It was removed; use application-level caching
Features added in MySQL 8?	Window functions, CTEs, roles, CHECK, invisible indexes, descending indexes
How to see a table's definition?	`SHOW CREATE TABLE t;`
How to see the execution plan?	`EXPLAIN` / `EXPLAIN ANALYZE`
What does DISTINCT do to performance?	Forces a sort or temporary table — avoid if not needed
UNION vs UNION ALL?	UNION removes duplicates (slower); UNION ALL keeps them
What is the ON clause vs USING?	`USING(col)` is shorthand when both tables share the column name
Fastest way to empty a table?	TRUNCATE
How to get the id you just inserted?	`LAST_INSERT_ID()`
How to see the number of rows affected?	`ROW_COUNT()`
What is a composite key?	A primary key made of two or more columns
What is referential integrity?	A foreign key value must exist in the parent table
What is MVCC?	Keeping old row versions so readers do not block writers

19. Output Prediction Questions (Tricky)

Interviewers love these because they expose shaky understanding of NULL and grouping.

P1. What does this return?

```
SELECT COUNT(*), COUNT(commission) FROM employees;
```

Answer: 6 and 2. `COUNT(*)` counts rows; `COUNT(column)` skips NULLs.

P2. What is the result?

```
SELECT 10 + NULL, CONCAT('a', NULL), NULL = NULL, NULL <=> NULL;
```

Answer: NULL, NULL, NULL, 1. Any arithmetic or CONCAT with NULL gives NULL. `=` with NULL gives NULL (not true). Only the NULL-safe operator `<=>` returns 1.

P3. Does AVG ignore NULLs?

```
SELECT AVG(commission) FROM employees;  -- (5000+3000)/2 = 4000
```

Answer: yes. AVG divides by the number of **non-NULL** values (2), not the number of rows (6). If you want the NULLs treated as zero, write `AVG(IFNULL(commission,0))`, which gives 1333.33.

P4. Why does this fail in MySQL 8?

```
SELECT dept_id, first_name, COUNT(*) FROM employees GROUP BY dept_id;
```

Answer: `first_name` is neither grouped nor aggregated, and `ONLY_FULL_GROUP_BY` is enabled by default, so MySQL raises an error. Older versions returned an arbitrary name silently. Fix it by adding the column to GROUP BY or wrapping it in `ANY_VALUE()` / `MAX()`.

P5. How many rows does this return?

```
SELECT * FROM employees, departments;
```

Answer: 6 × 3 = 18. Omitting the join condition creates a Cartesian product — a common accidental bug.

P6. What is the difference in these two counts?

```
SELECT COUNT(*) FROM employees e LEFT JOIN departments d
  ON e.dept_id = d.dept_id;                    -- 6
SELECT COUNT(*) FROM employees e INNER JOIN departments d
  ON e.dept_id = d.dept_id;                    -- 5
```

Answer: Ayesha has a NULL dept_id, so she is kept by the LEFT JOIN and dropped by the INNER JOIN.

P7. Which rows does this return if any dept_id in employees is NULL?

```
SELECT * FROM departments
WHERE dept_id NOT IN (SELECT dept_id FROM employees);
```

Answer: none at all. The NULL in the list makes the comparison unknown for every row. Use `NOT EXISTS` or add `WHERE dept_id IS NOT NULL` to the subquery.

P8. What will ORDER BY do with these values?

```
SELECT * FROM employees ORDER BY dept_id ASC;
```

Answer: NULLs come **first** in ascending order in MySQL, and last in descending order.

P9. Is this query valid?

```
SELECT salary * 12 AS annual FROM employees WHERE annual > 500000;
```

Answer: no — "Unknown column 'annual' in where clause". WHERE is processed before SELECT creates the alias. Either repeat the expression, or use HAVING (which runs after SELECT).

P10. What happens here?

```
SELECT 'abc' = 'ABC';
```

Answer: 1 (true), because the default collation ends in `_ci` meaning case-insensitive. To compare case-sensitively use a `_bin` collation or `BINARY 'abc' = 'ABC'`.

20. Final Preparation Checklist

Queries you must be able to write from memory

#	Problem	Key technique
1	Nth highest salary	LIMIT/OFFSET or DENSE_RANK
2	Top N per group	DENSE_RANK + PARTITION BY
3	Find duplicates	GROUP BY + HAVING COUNT(*) > 1
4	Delete duplicates keep one	Self join on id > id
5	Rows with no match	LEFT JOIN + IS NULL, or NOT EXISTS
6	Employee and manager	Self join
7	Above own group average	Correlated subquery or derived table join
8	Running total	SUM() OVER (ORDER BY ...)
9	Month-over-month change	LAG()
10	Pivot by status	SUM(CASE WHEN ...)
11	Bought all products	COUNT(DISTINCT) = total
12	Hierarchy / org chart	Recursive CTE

Concepts you must be able to explain in 30 seconds

- WHERE vs HAVING, and the logical execution order of a SELECT.
- INNER vs LEFT JOIN, and why a WHERE on the right table breaks a LEFT JOIN.
- Why `= NULL` never works and why NOT IN breaks with NULLs.
- ROW_NUMBER vs RANK vs DENSE_RANK.
- Clustered vs non-clustered index, and what a covering index is.
- Why an index speeds reads but slows writes.
- ACID, and what each letter means in practice.
- The four isolation levels and which problems each prevents.
- What a deadlock is and how to avoid it.
- 1NF, 2NF, 3NF in one sentence each, plus when to denormalize.
- DELETE vs TRUNCATE vs DROP.
- InnoDB vs MyISAM.
- How you would debug a slow query with EXPLAIN.

The day before

1. Re-read all the starred (★) questions in this book.
2. Write out the 12 must-know queries above on paper, without looking.
3. Practise saying the EXPLAIN debugging answer out loud — it comes up constantly.
4. Prepare one real story: a query or schema you actually improved, and by how much.

> **In the interview:** think out loud, ask one clarifying question before writing, format your SQL neatly on separate lines, and if you blank on syntax, describe the logic anyway. A clear explanation with a small syntax slip beats silence every time.

You are ready. Good luck!